

Sideflash

The sfl hybrid address protocol for Ark and Lightning

Whitepaper v0.1 | Proposed protocol | sfl1 addresses

Abstract

Sideflash gives a recipient one payment address containing a native Ark destination and a reusable BOLT12 offer. A compatible wallet uses native Ark when both parties use the same Ark service provider (ASP), and Lightning when they use different ASPs. The address starts with sfl1. The recipient continues to receive Ark value through either route.

Sideflash binds both routes to the same receiving authority. Its signed envelope carries the full destination ASP identity, Ark destination, offer and validity terms. A wallet extracts the offer rather than deriving it from the Ark key. Discovery verifies current service information; per-payment preparation binds the eventual invoice to Ark delivery.

Sideflash is a proposed application protocol above Ark and BOLT12. It is not a new asset or blockchain. Its address schema, authentication objects and delivery profile require implementation and review. This paper defines the protocol design and its validation requirements; it does not claim production readiness.

1 What the sender needs to learn

Question	Evidence
Which ASP serves this destination?	An ASP identity key pinned in the payment destination, plus an authenticated service descriptor.
Who controls the recipient destination?	A signature verifiable under the address's receiving policy, or a narrowly scoped delegation from that policy.
Which offer pays that destination?	A recipient-authorized mapping to exact offer bytes, acknowledged by the destination ASP.
Does this invoice belong to this payment?	Standard invoice validation plus a signed per-payment Ark delivery binding.

Here, ownership means control of the required cryptographic keys, not proof of a person's legal identity. A user may have multiple receiving addresses or use several ASPs. The protocol verifies the particular destination supplied by the sender rather than maintaining a global directory of people.

2 One address with two payment routes

A Sideflash envelope carries both the native Ark destination and the complete reusable BOLT12 offer. This is the recommended convenience profile: a compatible wallet decodes both routes from one address, uses native Ark within the same ASP and Lightning across ASPs. The resolution flow described later remains useful for fresh status, offer rotation and compact addresses that contain only a mapping reference.

Bind both routes to one destination

Embed the version, chain context, ASP identity, canonical Ark address, offer TLV bytes, mapping revision and validity interval. Include the recipient authorization and ASP acknowledgment described later, or their explicitly verifiable delegated forms. The authorization commits to the Ark destination and offer hash. An attacker must not be able to keep the visible Ark address while replacing the Lightning route.

The envelope needs its own specified encoding and checksum, with a bounded payload size. A new format must not masquerade as an existing native Ark address. Embedded offers and signatures make it larger, so QR capacity and scanning reliability are implementation constraints. A compact resolver-backed profile trades smaller addresses for an online lookup.

Deterministic route selection

Condition	Selected behavior
Matching chain and pinned ASP identity	Decode and pay the native Ark destination under the local receiving policy. No Lightning payment is needed.
Matching chain but different ASP identity	Verify the signed mapping and decode the embedded offer. Obtain an invoice and authenticated delivery binding before payment.
Unrecognized version or incompatible chain	Reject the destination rather than guessing a route.
Expired or revoked Lightning mapping	Resolve a fresh recipient-authorized mapping or ask for a new address. Never substitute an unrelated offer.

Compare cryptographic ASP identities rather than URL spelling or node aliases. The local receiving policy must be supported even when identities match. Do not fall back to Lightning after an uncertain local payment outcome: first establish that the original payment did not complete, then obtain approval for changed fees.

What decoding accomplishes

An embedded offer can be extracted without asking another ASP where to find it. It still requires invoice negotiation and delivery preparation. A wallet can verify the signed envelope offline, but cannot learn later revocation or present liquidity from those bytes alone. Cross-server payment therefore performs a freshness check with B before paying.

This format lets the user share one destination while preserving efficient local transfers. It does not guarantee support in older wallets, eliminate destination Ark inventory, or turn offer validation into proof of Ark delivery. Its principal benefit is clear, authenticated transport selection without manual invoice handling.

3 Address contents that can be resolved

Use a versioned Sideflash envelope containing the native Ark address, complete offer, both mapping signatures and routing information. The table below lists the core identity fields; the embedded offer, revision, validity interval, recipient authorization and ASP acknowledgment complete the envelope. Existing addresses must retain their existing interpretation. Do not append bytes to an established encoding and assume older wallets will understand them.

Proposed field	Meaning
version and required_features	Parsing rules and mandatory capabilities. Reject unsupported requirements.
chain_context	A precise chain and consensus profile identifier, not a ticker alone. Include genesis and a fork discriminator where needed.
ark_address_bytes	The canonical decoded native address, including its receiving policy.
asp_identity_key	The destination ASP's pinned public identity key.
resolver_hints	Bounded HTTPS origins or onion service locations; hints are locators, not identities.
recipient_binding_key	Optional separate key, accompanied by a delegation verifiable from the native receiving policy.

If the native address already commits to the ASP identity, compare it to the envelope and reject disagreement. If it contains only a short fingerprint, resolve a full key and apply the format's collision-security rules; do not silently treat a short identifier as a full-strength trust anchor.

Stable destination identity

Define `destination_id` as a domain-separated hash of the protocol version, `chain_context`, canonical Ark address bytes and pinned ASP identity. Endpoint URLs are excluded from this identifier so a service can relocate without changing the recipient. Signatures over endpoint descriptors still bind the allowed locations.

The sender's wallet parses the envelope and computes `destination_id` locally. Different textual representations of the same address must not produce different identifiers. It rejects invalid checksums where the native format defines them, malformed keys, unsupported policies and ambiguous encodings before making network requests.

Addresses without routing information

A legacy address may be payable only when the wallet already has a trusted ASP descriptor or the recipient supplies the new envelope. A server key does not reveal a hostname or Lightning offer. An unknown legacy destination must fail with a request for routing information, not trigger a guess or a search through unrelated servers.

4 Registration and recipient authorization

Bob's wallet first authenticates ASP B using the pinned identity. It then registers the receiving destination. A server account or API token may control access to registration, but it is not a substitute for proof that Bob controls the destination key.

Proof of control

B issues a single-use challenge containing a random nonce, `destination_id`, B's identity and a short expiry. Bob signs a registration-specific digest. For a simple public-key address, B verifies it against that receiving key. For multisignature or scripted policies, the profile must define a valid policy witness; unsupported policies are rejected.

A dedicated binding key can avoid repeatedly using a spending key. Its delegation must identify the binding key, exact destination, ASP, permitted mapping operations and expiry. It must not authorize spending funds or binding unrelated addresses. All delegated signatures are checked back to the receiving policy.

Approve the exact offer

B creates a reusable offer for this destination and returns its canonical TLV bytes to Bob. Bob verifies the network profile and signs a `MappingAuthorization` containing `destination_id`, ASP identity, `offer_bytes_hash`, revision, validity interval and the allowed delivery profile. B verifies Bob's signature, stores the association and countersigns the complete authorization.

Prefer one distinct offer per destination for the initial design. B persists the exact offer association with the destination. It must not infer the recipient from a display description, a payer note or an unvalidated client field. A generic ASP-wide offer would need a separate authenticated recipient-selection mechanism and is outside this initial profile.

What the signatures prove

Bob's signature means that the holder of the destination authority authorized that exact offer mapping. B's countersignature means that the pinned ASP acknowledges and serves it. Neither signature proves that B has enough liquidity or that Bob has already received a VTXO. Those are per-payment checks.

The registration challenge prevents replay of enrollment requests. The mapping authorization is intentionally reusable during its validity period. Signing domains must differ so a registration signature cannot be interpreted as an offer authorization or a transaction signature.

5 Discovery between independent ASPs

A does not need B to broadcast all of its users, and B does not need an account on A. Resolution happens on demand when Alice submits Bob's destination. Alice's wallet should independently verify the returned evidence even if ASP A performs the network requests.

Authenticate the destination service

A contacts a resolver hint using HTTPS, or an explicitly supported onion transport. It obtains a ServiceDescriptor signed by the identity key pinned in the envelope. The descriptor names the chain context, supported resolution versions, endpoints, validity period and any delegated online response keys. A rejects key or network mismatches.

TLS authenticates the contacted origin; the pinned ASP signature authenticates the protocol identity. A DNS answer or a new endpoint response must not replace that key. Rotation requires authorization by the previously trusted key or a newly verified payment destination; a replacement key cannot authorize itself.

Resolve the exact destination

A can extract the offer locally from sfl1. Before a cross-ASP payment, it sends destination_id, the envelope needed to check it, and a fresh request_nonce to B for a current signed status or replacement mapping. B returns the MappingAuthorization, recipient proof, offer bytes and a signed ResolutionResponse. That response commits to the request nonce, destination_id, mapping revision, mapping hash and a short expiration.

The nonce-bound response prevents a network intermediary from replaying an old response as fresh. It does not prevent a malicious ASP from equivocating. Clients should remember the highest accepted revision and reject rollbacks; new clients cannot infer unseen revocations without an additional trusted freshness mechanism.

Verification order

Step	Check performed by A and ideally Alice's wallet
1	Verify the service descriptor under the pinned ASP identity.
2	Match chain context, destination_id, nonce and validity windows.
3	Validate recipient authorization against the address policy or delegation.
4	Verify B's mapping acknowledgment and fresh response signature.
5	Decode the offer and compare the hash of exact canonical TLV bytes.
6	Enforce supported delivery capabilities and revision policy. Only then request an invoice.

For a Sideflash address, the mapping is embedded and verified. The network request establishes freshness; it is not needed to extract the initial offer. An ordinary Ark address without this envelope does not contain enough information to reconstruct a BOLT12 offer.

6 From a reusable offer to one payment

BOLT12 provides an offer, an invoice request and a returned invoice. The payer validates the invoice under BOLT12, including its relationship to the request. Offers can contain blinded paths; the application must not assume a visible offer issuer key is always present. The Ark mapping signature is a separate authorization layer. [1]

Bind the invoice to Ark delivery

For each payment, create a fresh `payment_intent_id`. Before A pays, require a signed `DeliveryBinding` from B containing `destination_id`, mapping revision, `offer_bytes_hash`, `invoice_bytes_hash`, `payment_hash`, Lightning amount in millisatoshis, recipient net amount in sats, recovery and service charges, reservation ID, expiry and `delivery_profile`.

A compares the actual invoice hash, amount and payment hash to that binding. It verifies the mapping again if it has expired. A then presents Alice with the net receipt, total source debit and maximum routing fee. The same destination may have many simultaneous intents; none may overwrite another's receive state.

Correlate without changing BOLT12 semantics

The initial profile can use an authenticated application side channel. A requests an invoice through the offer, then submits the exact invoice with a fresh intent identifier to B's preparation endpoint. B proves that this is one of its issued invoices, binds it to the mapped destination and returns the `DeliveryBinding` before payment. For invoices that are expired, unrecognized or already assigned, preparation fails.

B must gate successful settlement on a prepared delivery record. An invoice issued through the offer alone must not allow an unprepared payment to settle into an ordinary custodial balance. The invoice-request service and Ark receive engine therefore need an integrated state machine, not merely a static offer table.

Net amount versus invoice amount

Invoice value and recipient spendable value are different quantities when receive costs apply. A quote can state the invoice amount required to deliver a requested net amount, but the final invoice and `DeliveryBinding` must match that quote. Amounts use bounded integers with explicit units. If inventory or fees change, B rejects preparation and requests a new quote rather than silently reducing Bob's receipt.

BOLT12 does not by itself standardize the Ark destination binding, inventory reservation or delivery proof. These are mandatory parts of the proposed application protocol. Do not place essential security data in a free-form payer note and assume Lightning implementations enforce it.

7 Settlement and recipient protection

Correct resolution establishes where a payment is intended to go. It does not make an Ark credit atomic with Lightning settlement. The delivery profile must separately specify how Bob obtains an enforceable conditional VT XO before the payment becomes irreversible.

Prepare before settling

B reserves destination Ark inventory and constructs the conditional receive for Bob. Its signed delivery proof must commit to the correct recipient, amount, payment hash, recovery conditions and funding ancestry. Bob or an authorized wallet agent validates and durably stores the necessary material. A retains an authenticated delivery acceptance or the independently verifiable proof required by the selected profile.

A locks Alice's Ark spend under the matching payment conditions and initiates Lightning payment. Settlement is allowed only when the profile's claim conditions are satisfied. On interruption, both services reconcile persisted state and Lightning status rather than creating a replacement invoice blindly.

Secret ownership must be explicit

A strict recipient-controlled profile requires Bob to control the payment preimage until his recovery package is valid. A stock offer service that generates and retains its own preimages cannot automatically provide that property. It needs a supported way to construct the invoice around an agreed payment hash and coordinate the claim. This is an integration requirement, not a standard feature supplied by the address mapping.

An existing Ark receive protocol may use a different cryptographic construction. It must specify its trust assumptions and prove the corresponding delivery guarantee. A server's statement that it credited an account is insufficient. The first interoperable profile should require Bob's wallet or an authorized agent online if safe offline delivery has not been demonstrated.

Separate payment and delivery state

Record	Example states
Lightning attempt	Created, in flight, succeeded, failed or unresolved.
Ark receive	Reserved, conditional proof ready, claimable, claimed, expired or refunded.
Application intent	Prepared, payment submitted, delivery pending, complete or failed.

A Lightning preimage proves invoice settlement under the payment protocol; it is not alone proof of the intended Ark credit. Mark the application intent complete only after the required delivery evidence is available. If Lightning succeeded and delivery remains pending, investigate or resume the claim; never automatically pay again.

8 Wire profile and failure handling

Deterministic signed objects

For this proposal, use deterministic CBOR with integer field keys, definite lengths, shortest encodings, no duplicate keys and no floating-point amounts. Publish exact field numbers and test vectors before implementation. Human-readable JSON examples must not become the signed representation. RFC 8949 supplies the encoding framework. [3]

Sign a domain-separated hash of each object's canonical bytes, excluding its signature field. Use a distinct domain for service descriptors, challenges, delegations, mapping authorizations, resolution responses and delivery bindings. A BIP340 Schnorr profile can use 32-byte x-only keys and 64-byte signatures; it is separate from transaction signing rules. [2]

Hash decoded offer and invoice TLV bytes rather than case or line wrapping of their textual forms. Follow the Lightning parser's validity rules and preserve all relevant bytes. Do not normalize away unrecognized fields before hashing. Expiry timestamps, nonces, keys and maximum object sizes require explicit bounds.

Suggested application endpoints

Illustrative endpoint	Purpose
GET /.well-known/ark-resolution	Fetch signed service descriptor.
POST /v1/resolve	Return recipient authorization and fresh signed mapping.
POST /v1/prepare	Bind an issued invoice to a reserved Ark delivery intent.
POST /v1/status	Return authenticated payment and delivery state for an authorized intent.

These paths are proposed conventions. They are not existing Ark or BOLT12 endpoints. Registration, revocation and quote APIs also need authorization, rate limits and defined error responses.

Reject ambiguity rather than redirect funds

Return explicit errors for unknown destination, revoked mapping, unsupported chain, expired quote, insufficient inventory and invalid invoice. Never fall back to a default ASP offer. Offer rotation requires new recipient authorization; outstanding intents retain their original immutable mapping and invoice binding until resolved.

For a request retry, reuse payment_intent_id and require the same parameters. Changed parameters under the same ID are rejected. Routing retries against the same invoice are distinct from issuing a new invoice. A failed HTTP response cannot establish whether Lightning settled; reconciliation must consult durable records.

9 Privacy liquidity and validation

Do not turn resolution into an address directory

Avoid public listing APIs. Use high-entropy receiving destinations, optional read capabilities, limited response sizes and rate limits. A reusable destination remains linkable to parties who see it, and B necessarily learns which receive it services. Blinded Lightning paths do not conceal information already disclosed to the resolver.

Treat endpoint hints as hostile network input. Disallow credentials in URLs, unapproved schemes and arbitrary redirects. Public services should block loopback, link-local, metadata and private-network destinations, including after DNS resolution. Private deployments need an explicit allowlist; onion access should use an isolated transport policy. Do not let an address turn ASP A into a network probe.

Liquidity still has to be managed

A pays from Lightning liquidity and collects Alice's Ark-side value. B receives Lightning value and delivers existing Ark inventory to Bob. No separate cross-ASP swap provider is required, but B still needs usable Ark funding. This proposal is address interoperability and settlement routing; it does not move Lightning channel backing directly into Ark.

Quotes must cover the recipient's net amount, receive allocations, service costs and a bounded routing allowance. Rebalancing and failed-attempt costs must be priced or otherwise limited. Insufficient inventory must be detected before settlement, not handled by issuing an unsecured credit.

Interoperability tests

Test correct resolution, substituted endpoints, wrong ASP keys, wrong recipient signatures, delegated-key expiry, offer substitution, stale revisions, fork-context mismatch and malformed encodings. Test a valid invoice for a different destination, changed invoice amounts, simultaneous receives, reused intent IDs, reservation expiry and missing delivery proofs. Include crashes immediately before and after each durable commit and a successful Lightning payment whose HTTP response is lost.

The decisive acceptance test is that Alice can submit one destination from B, verify the entire identity chain through A, approve an exact cost, and obtain evidence that Bob can claim the intended Ark value. Resolving the right offer is necessary, but the delivery protocol must be validated independently.

Standards referenced

[1] BOLT 12 - Negotiation Protocol for Lightning Payments

[2] BIP 340 - Schnorr Signatures for secp256k1

[3] RFC 8949 - Concise Binary Object Representation

Referenced for their respective primitives. The Sideflash envelope, identity hierarchy and application messages in this paper are proposals, not additions already adopted by those standards.

10 Encoding and compatibility profile

The sfl prefix

The human-readable prefix is sfl; the following 1 is the Bech32 separator, not a version number. A displayed address has the form sfl1 followed by encoded payload and checksum. This notation is illustrative and is not a payable example. Version and chain identity are explicit fields inside the signed payload.

Use a Bech32m checksum for the outer representation. Writers use lowercase; readers may accept an entirely uppercase form but reject mixed case. The checksum detects transcription errors, while signatures authenticate the mapping. Prefix recognition alone authenticates neither the recipient nor the network.

Bounded binary payload

The proposed payload is deterministic CBOR containing version, required features, chain context, full ASP identity, native address bytes, exact decoded offer TLV bytes, mapping revision, validity interval, recipient authorization and ASP acknowledgment. Resolver hints and delegated-key certificates are optional bounded fields. Assign integer keys and publish canonical byte-level test vectors before declaring a stable encoding.

A proposed first profile limits decoded payloads to 2,048 bytes and input text to 4,096 ASCII characters, checking text length before decoding. It must explicitly support longer envelopes than the original Bech32 address-length limit. Libraries that enforce a shorter limit must reject clearly rather than truncate. Large blinded offers or certificate chains may exceed this profile and require a fresh, smaller address; oversized input is never silently accepted.

Signing and version transitions

The recipient signs a domain-separated authorization covering both destinations, ASP identity, network, revision, validity and delivery profile. The ASP acknowledgment signs that authorization including its recipient signature. Bind resolver hints in an authenticated descriptor or signed envelope extension. Neither party can replace the offer without invalidating the applicable signatures.

A separately authenticated recipient channel establishes that the supplied address is Bob's. Internal signatures prove consistency and key control; they cannot detect wholesale replacement with an attacker's valid address. Preserve trusted identity pins and clearly surface identity changes.

Sideflash implementations must agree on exact field encodings, signature domains and feature negotiation before exchanging addresses. Future versions require explicit migration rules. Unknown versions or required features fail closed; ordinary Ark and BOLT12 formats remain unchanged.

Minimum useful release

A complete first release needs address issuance, recipient and ASP signing, strict decoding, both route selectors, authenticated cross-ASP preparation, fee approval and delivery reconciliation. Tests must demonstrate a local native payment and a remote Lightning-backed Ark payment from the same Sideflash address, including failures. Codec success alone is not payment interoperability.